

情感分析实验报告

2024010694 王珺桥

代码位于: <https://cloud.tsinghua.edu.cn/d/336c2aa45f1b4a51b0e7/>

1. 实验概述

本实验基于给定数据集，实现了CNN、BiLSTM、MLP和attention模型，均达到80%以上准确率。其中MLP模型在初始实验中表现最佳，说明在该数据集上，情感分类主要依赖局部语义信息，而非复杂的序列结构。在实验过程中，进一步对学习率、dropout和训练轮数等关键参数进行了调优，分析了不同模型对参数变化的敏感性。结果表明，BiLSTM和Attention模型对参数较为敏感，而CNN和MLP表现相对稳定，说明模型复杂度与调参需求之间存在明显关系。

同时，基于接入的deepseek-V4生成的数据，对四个模型也进行了测试，均达到80%以上准确率，具有一定鲁棒性。在大模型生成的数据上，各模型性能均显著提升，说明该数据在语义表达上更加规范，降低了分类难度。同时，这也反映出模型性能对数据分布具有较强依赖性。

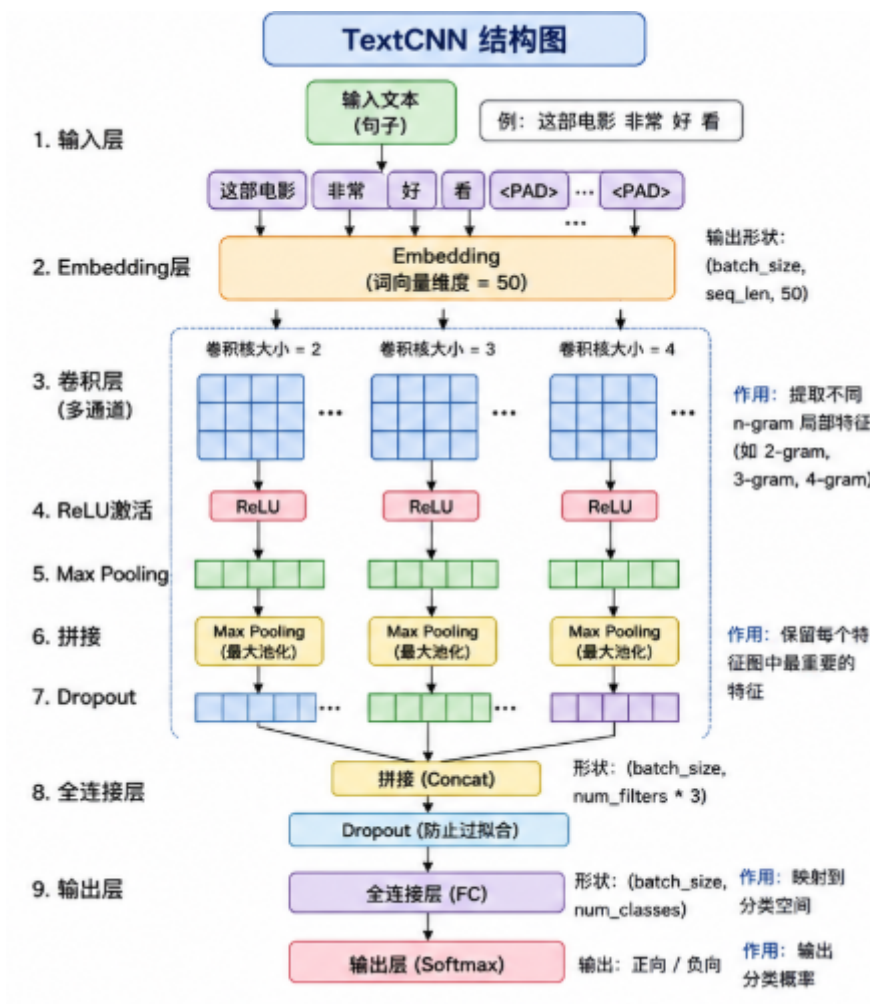
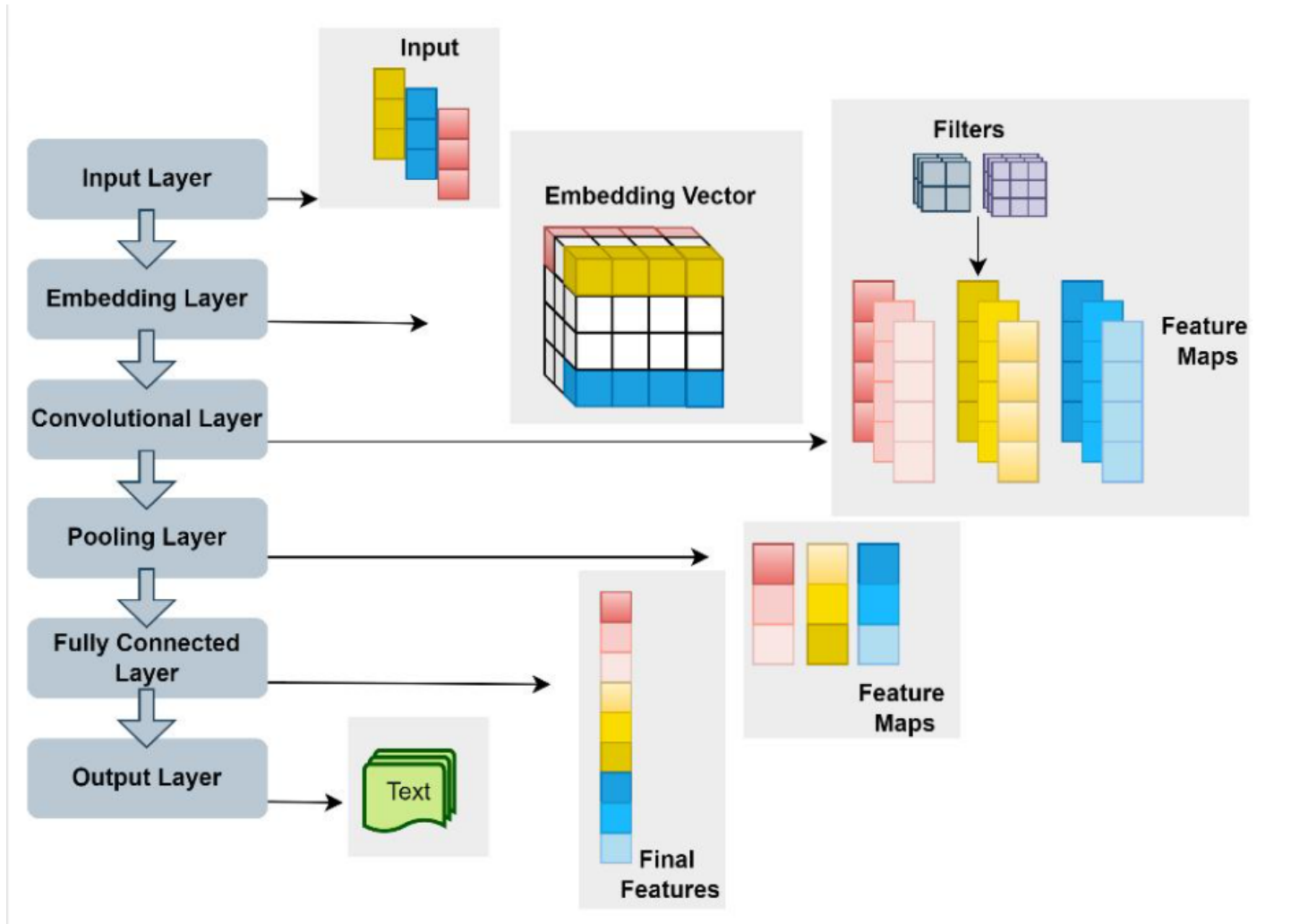
在代码设计中，主要包括数据处理、模型定义、训练流程和结果分析四个部分。其中，data_trans.py负责数据读取、词表构建以及文本向量化处理；models.py定义了CNN、BiLSTM、MLP和Attention四种模型结构；train.py实现模型训练、验证以及评价指标计算；main.py作为程序入口，负责组织整个训练流程。

此外，本实验实现了实验日志记录机制，logger.py用于保存每次实验的参数配置和测试结果，将每次训练结果和参数保存到独立目录中，便于后续对比分析。同时，plot_results.py提供可视化功能，可以对dropout的多次调参结果进行绘图分析。

本实验斥1元巨资，购入deepseek-V4API keys的token（笑

2. 结构图

2.1 CNN



- 模型结构：输入文本 → embedding → 多通道卷积 (2,3,4) → ReLU → MaxPooling → 拼接 → dropout → 全连接层 → 输出

embedding层：将词ID转换为向量表示

卷积层：提取局部 n-gram 特征

ReLU：引入非线性，增强表达能力

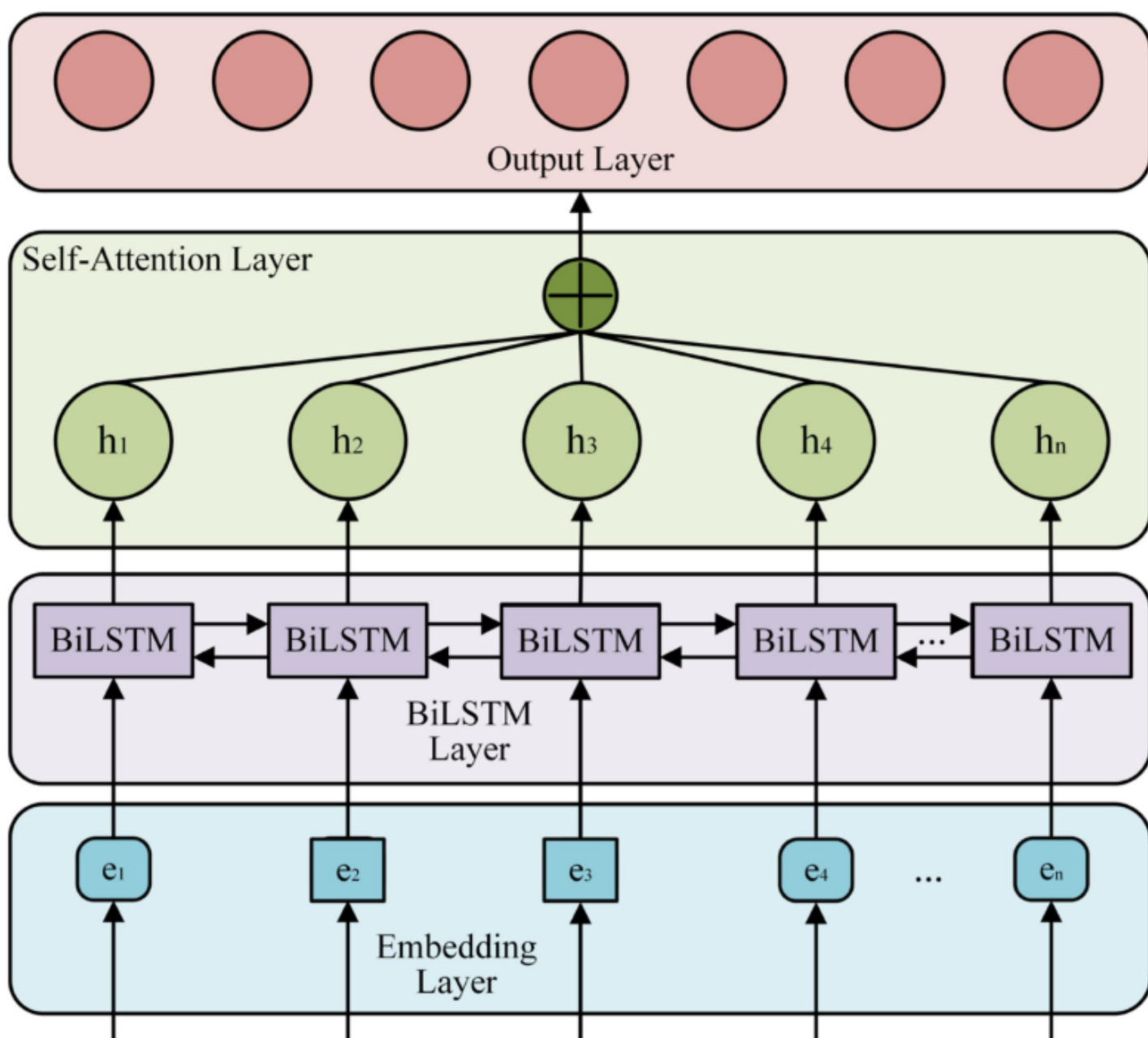
MaxPooling：从每个特征图中提取最重要的特征

拼接：融合不同 n-gram 信息，2-gram + 3-gram + 4-gram → 综合表达

dropout：防止过拟合

全连接层：将特征映射到分类空间

2.2 RNN



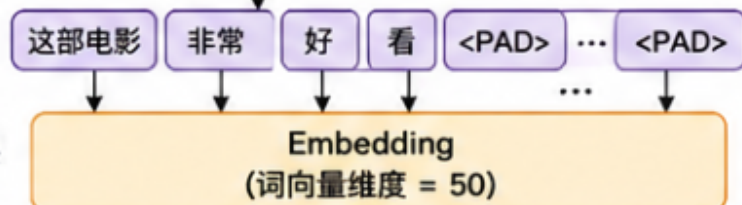
BiLSTM (RNN) 结构图

1. 输入层

输入文本
(句子)

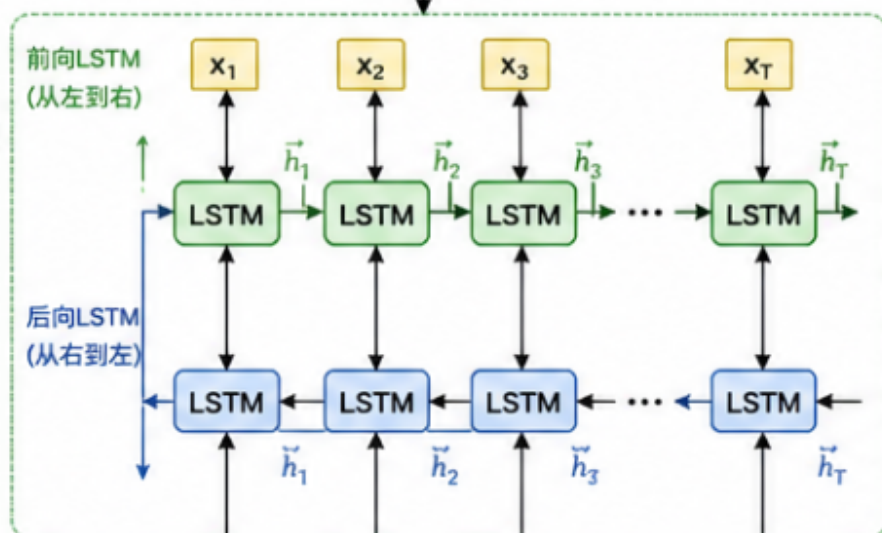
例: 这部电影 非常 好看

2. Embedding层



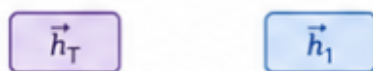
输出形状:
(batch_size,
seq_len, 50)

3. BiLSTM层



作用:
同时捕捉前向
和后向的上下文
信息

4. 取最终隐藏状态



拼接 (Concat)

形状: (batch_size,
hidden_size * 2)

作用: 得到整句
的语义表示

5. Dropout

Dropout (防止过拟合)

6. 全连接层

全连接层 (FC)

形状: (batch_size,
num_classes)

作用: 映射到
分类空间

7. 输出层

输出层 (Softmax)

输出: 正向 / 负向

作用: 输出
分类概率

- 模型结构: 输入文本 → embedding → BiLSTM → 取最后隐藏状态 → 拼接 → dropout → 全连接层 → 输出

embedding层: 将词ID转换为向量表示

LSTM层: 建模序列信息 (上下文关系), 内部为输入门 + 遗忘门 + 输出门

BiLSTM: 输出hidden_size × 2, 同时利用前后文信息

取最后隐藏状态: 得到整句的语义表示

dropout：防止过拟合

全连接层：输出分类结果

3. 实验方法

3.1 实验流程

整体流程包括数据加载、数据预处理、模型训练和结果评估四个阶段。

首先，程序读取train.txt、validation.txt和test.txt三个数据文件。每条样本包含情感标签和已经分词的文本，其中标签0表示负向情感，标签1表示正向情感。读取数据后，程序根据训练、验证和测试文本构建词表，并加入<PAD>和<UNK>两个特殊符号，分别用于句子补齐和未知词处理。

随后，利用预训练的词向量文件构建embedding矩阵，将词表中的词映射为 50维向量。对于 Word2Vec中存在的词，使用预训练向量初始化；对于不存在的词，使用随机初始化。之后，将文本转换为词 ID 序列，并根据最大长度max_len进行截断或 padding，最终通过Dataset和DataLoader构造批量训练数据。

模型训练阶段分别训练 CNN、BiLSTM、MLP 和 Attention 模型。训练时使用交叉熵损失函数 CrossEntropyLoss和Adam优化器，并在每轮训练后使用验证集计算 Accuracy、Precision、Recall 和 F1-score。程序通过验证集 F1-score 选择最佳模型参数，并使用Early Stopping防止过拟合。

最后，在测试集上评估最佳模型，并输出 Accuracy、Precision、Recall、F1-score 和 Loss。同时，实验结果会被自动保存到以时间命名的日志目录中，便于后续对不同参数设置和不同数据集实验进行对比分析。

3.2 模型结构

本实验实现了四种模型：

- TextCNN：使用不同尺寸卷积核（2,3,4）提取n-gram特征，通过最大池化提取最显著特征，最终通过全连接层分类
- BiLSTM：双向 LSTM 建模文本序列，拼接正向与反向隐藏状态作为句子表示
- MLP：对词向量进行平均，使用全连接网络分类
- Attention：为每个词分配权重，通过加权平均得到句子表示

3.3 训练方法

初始参数为


```
{
    "data_dir": "./Dataset",
    "seed": 42,
    "epochs": 5,
    "batch_size": 32,
    "max_len": 80,
    "min_freq": 1,
    "lr": 0.001,
    "weight_decay": 1e-05,
    "dropout": 0.5,
    "patience": 4,
    "cpu": false
}
```

- 损失函数：CrossEntropyLoss
- 优化器：Adam
- 评价指标：Accuracy / Precision / Recall / F1-score
- 使用 Early Stopping 防止过拟合

这是初始实验数据，调参后的结果后续呈现

4. 实验结果

4.1 模型性能

初始结果和参数存储于results/2026-05-02_20-37

模型	Accuracy	Precision	Recall	F1
CNN	0.8455	0.8495	0.8449	0.8472
BiLSTM	0.813	0.8642	0.7487	0.8023
MLP	0.8537	0.8982	0.8021	0.8475
Attention	0.8455	0.8611	0.8289	0.8447

4.2 结果分析

MLP 表现最佳，说明预训练embedding已包含较强语义信息；CNN 表现稳定，说明局部特征对情感分类有效；BiLSTM 表现较差，可能存在过拟合问题；Attention 提升了模型对关键特征的关注能力。

该数据集为情感分类任务，文本长度较短，情感表达通常依赖于局部关键词或短语，而不依赖复杂的长距离上下文关系。在这种情况下，任务本质上更接近关键词分类，而非需要建模句子结构或语义关系的复杂序列任务，不需要长距离依赖，故RNN优势发挥不出来。而CNN 模型通过卷积提取局部 n-gram 特征，在该任务中表现与 MLP 接近，进一步说明该数据集主要依赖局部特征而非全局结构。

5. 参数分析

实验中主要调节参数包括：learning rate、dropout、epochs

- Learning rate：调参结果存储于results/2026-05-05_15-28

代码块

```
1  CUDA_VISIBLE_DEVICES=0 python model/main.py --epochs 5 --batch_size 32 --lr 0.0005 --dropout 0.5
```

模型	Accuracy (0.001)	Accuracy (0.0005)	ΔAcc	F1 (0.001)	F1 (0.0005)	ΔF1
CNN	0.8455	0.8509	+0.0054	0.8472	0.8541	+0.0069
BiLSTM	0.8130	0.8401	+0.0271	0.8023	0.8384	+0.0361
MLP	0.8537	0.8591	+0.0054	0.8475	0.8602	+0.0127
Attention	0.8455	0.8157	-0.0298	0.8447	0.8111	-0.0336

- 在初始实验中使用学习率 0.001 时，部分模型（尤其是 BiLSTM）表现不稳定，出现验证集指标波动较大的情况。学习率调整至0.0005后，BiLSTM 模型提升最为显著，F1-score 提升约 0.036，说明较小学习率有助于复杂模型更加稳定地收敛，从而显著提升性能；CNN 与 MLP 均有小幅提升（约 0.005~0.01），说明较低学习率对简单模型也有一定优化作用，但影响较为有限；Attention 模型性能明显下降，说明其对学习率较为敏感，较小学习率可能导致其训练不足或收敛到较差的局部最优。

总体来看，降低学习率能够显著提升模型稳定性，尤其对参数较多的序列模型（如 BiLSTM）效果最为明显。

- dropout：

代码块

```
1  CUDA_VISIBLE_DEVICES=0 python model/main.py --epochs 8 --batch_size 32 --lr 0.0005 --dropout 0.6
```

Results will be saved to: results/2026-05-02_21-08

代码块

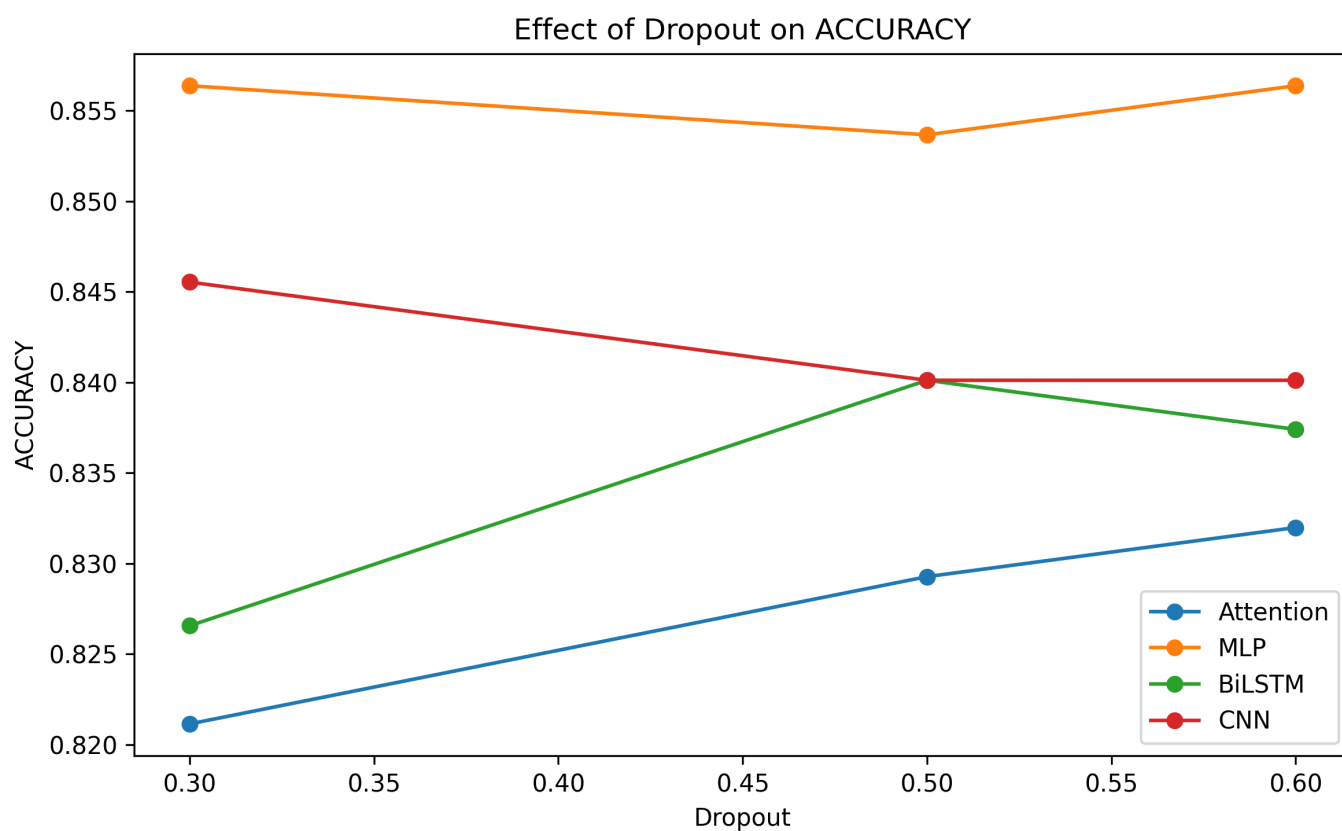
```
1 CUDA_VISIBLE_DEVICES=0 python model/main.py --epochs 8 --batch_size 32 --lr 0.0005 --dropout 0.5
```

Results will be saved to: results/2026-05-02_21-34

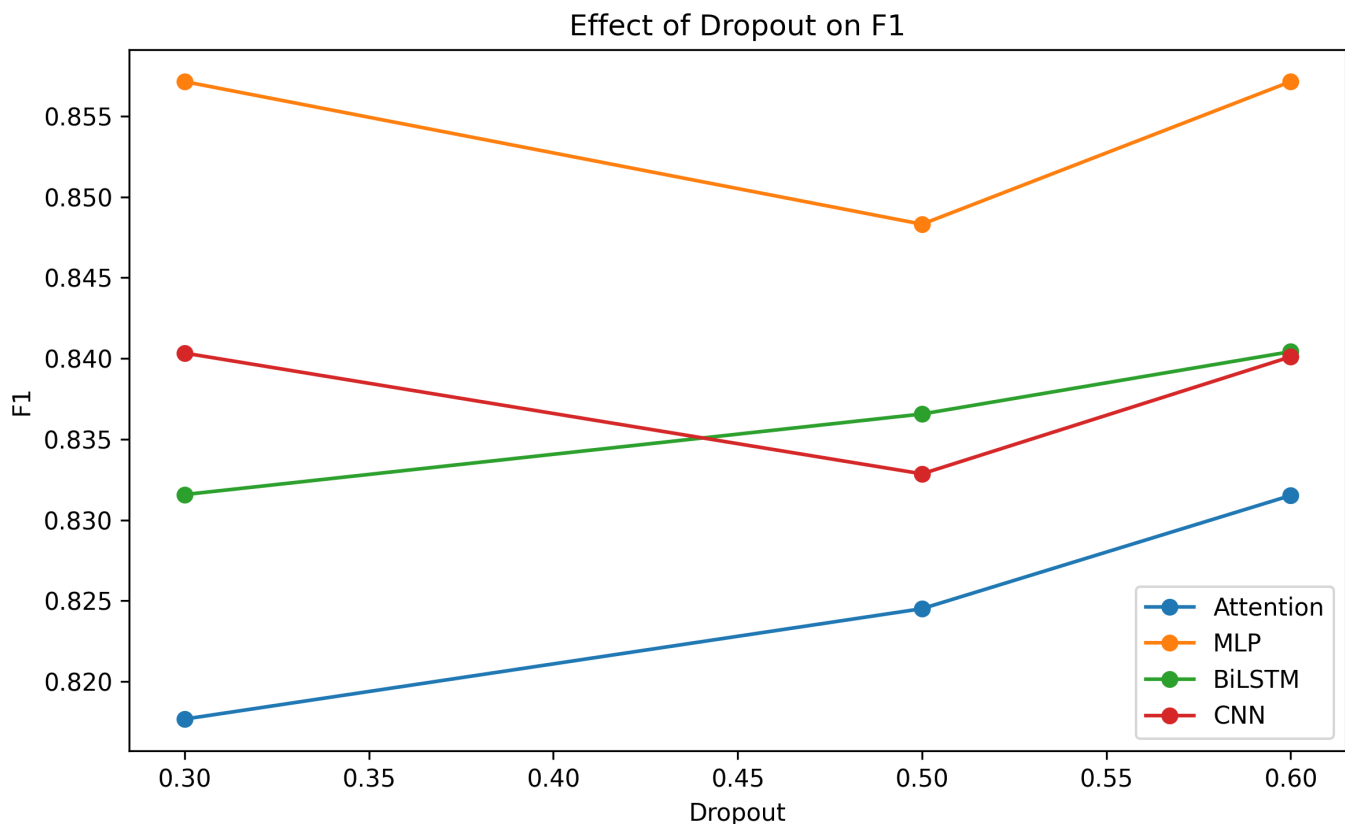
代码块

```
1 CUDA_VISIBLE_DEVICES=0 python model/main.py --epochs 8 --batch_size 32 --lr 0.0005 --dropout 0.3
```

Results will be saved to: results/2026-05-02_21-40



- 从 Accuracy 曲线可以观察到：（1）BiLSTM 和 Attention 随 dropout 增大整体提升，说明这两类模型对正则化需求较高；（2）CNN 和 MLP 表现较为稳定甚至略有下降，说明其在当前任务中不过拟合问题较轻；（3）BiLSTM 在 dropout=0.5 时达到最佳性能，说明适中的正则化最有利于其泛化能力。



- 从 F1-score 曲线可以观察到：（1）BiLSTM 和 Attention 随 dropout 增大持续提升，说明正则化能够显著改善其类别平衡能力；（2）MLP 表现稳定，说明其对 dropout 不敏感；（3）CNN 在中等 dropout（0.5）时性能下降，说明其对正则化程度较为敏感。
- 总体来看，F1-score 的变化趋势与 Accuracy 基本一致，但在细节上更加明显地反映了模型对类别不平衡的适应能力。
- epochs：调参结果存储于results/2026-05-05_15-25

代码块

```
1 CUDA_VISIBLE_DEVICES=0 python model/main.py --epochs 8 --batch_size 32 --lr 0.001 --dropout 0.5
```

模型	Accuracy (5)	Accuracy (8)	Δ Acc	F1 (5)	F1 (8)	Δ F1
CNN	0.8455	0.8401	-0.0054	0.8472	0.8418	-0.0054
BiLSTM	0.8130	0.8347	+0.0217	0.8023	0.8320	+0.0297
MLP	0.8537	0.8482	-0.0055	0.8475	0.8418	-0.0057
Attention	0.8455	0.8428	-0.0027	0.8447	0.8466	+0.0019

- 将epochs提高至8，BiLSTM 模型性能显著提升，其 F1-score 由 0.8023 提升至 0.8320，说明增加训练轮数有助于序列模型更充分地学习上下文信息，提高表达能力。Attention 模型略有提升（0.8447 → 0.8466），说明适当增加训练轮数对其有一定帮助，但提升幅度有限。CNN 与 MLP 模型性能略有下降，F1-score 分别从 0.8472 和 0.8475 降至约 0.8418，说明这两类模型在较少训练轮数下已经基本收敛，继续训练可能导致轻微过拟合。总的来讲增加训练轮数对复杂模型（如 BiLSTM）有明显收益，但对结构简单的模型（如 CNN 和 MLP）可能带来过拟合风险，因此应根据模型复杂度合理选择训练轮数。

总体来看，学习率和 dropout 是影响模型性能的关键参数。在不同模型中，MLP 表现最为稳定，对参数变化不敏感；CNN 次之；BiLSTM 对参数最为敏感，但在调参后提升最明显；Attention 模型表现波动较大。

6. 问题思考

6.1 训练何时停止最合适

本实验采用 Early Stopping 机制，即每轮训练后在验证集上计算 F1-score，如果验证集 F1-score 得到提升，则保存当前模型参数；如果连续若干轮没有提升，当未提升次数达到patience设置值时，提前停止训练。

具体实现方式为：设置best_val_f1记录当前最佳验证集F1-score，设置patience_count记录连续未提升轮数。当某一轮的验证集F1-score 高于历史最佳值时，更新最佳模型并将patience_count清零；否则将patience_count加一。当patience_count>=patience时停止训练。最终测试时，使用验证集表现最好的模型参数，而不是最后一轮参数。

固定epoch的方法简单但如果这个固定的次数过小，那么模型还没有学习完，不足以达到最佳；如果这个固定的次数太大，模型在固定迭代次数之前已经过拟合了，此时才停止训练，可能导致模型无法充分收敛和泛化。

通过验证集调整的方法可以根据模型在验证集上的实际性能来决定停止训练，更具灵活性。而且，这样能够自适应地判断什么时候开始过拟合，及时止损，获得更好的泛化能力。但缺点是需要每个训练步骤后计算模型在验证集上的性能，增加了额外的计算开销。

6.2 参数初始化方法

本实验中：词向量层使用预训练 Word2Vec 初始化。具体来说，对于词表中能够在预训练的词向量文件中找到的词，直接使用其 50 维预训练向量；对于未出现在预训练词向量中的词，则使用随机高斯分布初始化；<PAD>的向量设置为全零，避免 padding 对模型产生语义影响。CNN、BiLSTM、MLP 和 Attention 中的其他网络层则使用 PyTorch 默认初始化方式。

其他常见方法：

- 高斯初始化：适用于一般网络。通过从高斯分布中随机采样，可以打破权重的对称性，并使得每个神经元接收到的初始化权重具有一定的随机性，有助于加速网络的收敛速度和学习过程。

- Xavier初始化：适用于 CNN / MLP。能够在前向传播和反向传播中保持较稳定的方差，适合使用 Sigmoid、Tanh 等激活函数的网络。
- 正交初始化：适用于RNN。它的目标是使得权重矩阵是正交的。具体实现上可以通过对某个随机矩阵 A 进行 QR 分解后，得到 Q 矩阵即为一正交矩阵。在 RNN 中，正交初始化可以帮助缓解梯度消失和梯度爆炸问题。

6.3 如何防止过拟合

本实验采用：Dropout、Early Stopping、L2正则和增加数据量的方法。

- 通过调节 dropout 参数（0.3、0.5、0.6），观察其对模型性能的影响。实验结果表明，适当增大 dropout（如 0.5）能够有效提升模型在验证集上的表现，说明其能够缓解过拟合问题。但过大的 dropout（如 0.6）在部分模型（如 CNN）中会导致性能下降，说明过强的正则化会影响模型学习能力。
- Early Stopping 是通过监控验证集指标（如 F1-score），在模型性能不再提升时提前终止训练。本实验中，当验证集 F1-score 在连续若干轮（patience）内未提升时停止训练，并保存最佳模型参数。该方法能够有效避免模型在后期训练中对训练数据过拟合，同时减少不必要的训练时间。
- 在优化器中引入了 weight_decay（L2 正则化），通过对模型参数施加惩罚项，限制参数过大，从而降低模型复杂度。L2 正则化能够使模型权重更加平滑，从而提升模型在未见数据上的泛化能力。
- 通过使用大模型生成额外数据，在一定程度上丰富了数据分布，使模型能够学习更多样化的表达，从而提升泛化能力。

此外，还可以进行数据增强，通过对原始数据进行变换（如同义替换、句式变换等），增加训练数据的多样性，从而提升模型的泛化能力。在文本分类任务中，数据增强可以有效减少模型对特定表达方式的依赖。

6.4 模型对比分析

模型	优点	缺点
CNN	提取局部特征，计算快，可以很容易地通过添加更多的卷积层、池化层和全连接层来构建更深层次的模型，以适应不同复杂度的文本分类任务。	处理文本时无法直接建模长期依赖关系
RNN	捕捉上下文信息，可以处理变长的序列数据，对于序列建模任务效果较好	训练慢，易过拟合；有可能出现梯度消失或梯度爆炸的问题
MLP	简单高效，利用GPU加速时表现较好	忽略顺序且参数量较大，特别是在处理高维数据时容易导致过拟合

对于初始结果，MLP模型取得了最理想的效果。在本数据集上情感分类主要依赖局部关键词，而非长距离依赖，预训练词向量已提供较强语义表示，且数据规模有限，复杂模型更容易过拟合；因此，结构简单、训练稳定的 MLP 模型反而最优。当文本更长、包含转折、否定或复杂语义结构时，RNN（BiLSTM）通常会优于MLP。

这表明，模型性能不仅取决于模型复杂度，更取决于模型结构与数据分布之间的匹配程度。

6.5 模型鲁棒性分析

本实验接入DeepSeek-V4生成多样化情感文本，模拟更复杂的语言分布。

- a. 使用大模型 API 自动生成测试数据，生成规则为：
 - 1. 二分类任务（0=负向，1=正向）；
 - 2. 格式：标签 \t 分词句子；
 - 3. 词之间使用空格分隔；
 - 4. 数据类型均衡，包含：简单情感句、否定句、转折句、口语表达
 - b. 生成规模：
 - 小规模测试集：40条（robust_test(40).txt）
 - 大规模测试集：1000条（robust_test_large.txt）
- 最终数据划分为
- 训练集（Train）：大模型生成数据的 80%
 - 验证集（Validation）：大模型生成数据的 20%
 - 测试集（Test）：1. 原始 test.txt（标准测试）；2. robust_test(40).txt（鲁棒性测试）；3. robust_test_large.txt（大规模鲁棒性测试）

- c. 在保持 epochs=5、batch_size=32、lr=0.001、dropout=0.5 不变的情况下，仅将数据集由原始数据集替换为大模型生成数据集，从而分析数据分布变化对模型性能的影响

代码块

```
1  CUDA_VISIBLE_DEVICES=0 python model/main.py --data_dir ./Dataset_llm --
    epochs 5 --batch_size 32 --lr 0.001 --dropout 0.5
```

模型	Accuracy（原始）	Accuracy（LLM）	F1（原始）	F1（LLM）
CNN	0.8455	0.9000	0.8472	0.8947
BiLSTM	0.8130	0.9250	0.8023	0.9189
MLP	0.8537	0.8750	0.8475	0.8649
Attention	0.8455	0.9250	0.8447	0.9231

- 实验结果表明，在大模型生成的数据上，各模型性能均出现明显变化，且整体表现优于原始数据集：

（1）BiLSTM 和 Attention 模型提升最为显著，F1-score 分别从 0.8023 提升至 0.9189、从 0.8447 提升至 0.9231。BiLSTM 属于序列建模模型，能够捕捉文本中的长距离依赖关系和上下文信息。在大模型生成的数据中，文本通常语法规则、句式完整、上下文关系清晰，这使得 BiLSTM 可以更充分地利用其双向建模能力，从而获得更显著的性能提升。而 Attention 模型通过为不同词分配权重，能够动态聚焦于对情感判断最关键的词语。在生成数据中，情感表达通常更加明确，这使得 Attention 机制能够更容易地识别关键信息，从而提升分类效果。

（2）CNN 与 MLP 也有一定提升，但幅度较小，说明其对数据分布变化的适应能力相对有限。

整体来看，所有模型在 LLM 数据上的 Accuracy 和 F1-score 均显著高于原始数据，说明该数据集更容易分类。进一步分析认为原因可能有三点：（1）大模型生成的数据通常语法规则、情感表达明确，减少了原始数据中存在的歧义、噪声和非标准表达；（2）生成数据往往具有更清晰的情感边界，例如正负情感更加明显，从而降低模型判别难度；（3）相比真实数据，大模型生成的数据分布更加理想化，因此模型在该数据上的表现更高。

7. 实验总结与心得

通过本实验，我掌握了文本分类任务的完整流程，包括数据处理、模型设计和实验分析。实验结果表明，在当前任务中，CNN 和 MLP 表现较为稳定，而 BiLSTM 模型在数据规模较小时容易过拟合。引入 Attention 后，模型能够自动关注重要词语，提高了解释性。模型性能不仅取决于模型复杂度，更取决于模型结构与数据分布之间的匹配程度。

同时我重温了调用 api keys 的过程，用调用的 deepseek-V4 生成的数据集测试了模型的鲁棒性。

能把学习到的东西在作业中进行实践是很开心也很奇妙的经历，不仅是上课学到的CNN、RNN模型等，在srt项目中学到的关于跑实验的小技巧，比如tmux、实验日志、画图脚本等都在这次实验中再次亲自尝试，maybe以后有机会在作业中试一下消融实验？总之仍然有趣，仍然开心。